

CSS, Tailwind, React Components

Petr Wehrenberg

"CSS is easy." - said no one, ever

Outline

1. How can HTML be styled?
2. CSS basics - selectors, specificity, inheritance
3. Styling vs. Layouting
4. Flexbox & Grid explained
5. Tailwind CSS
6. Tailwind utilities (cn, clsx, twMerge)
7. CSS approaches compared
8. Component libraries & Shadcn
9. React component patterns
10. useState and useEffect

How can HTML be styled?

Three ways to add styles to HTML elements.

1. External stylesheet

```
/* styles.css */  
.cool-navbar {  
  display: flex;  
  color: #343434;  
}
```

```
<html>  
  <head>  
    <link rel="stylesheet" href="styles.css" />  
  </head>  
  <body>  
    <nav class="cool-navbar">...</nav>  
  </body>  
</html>
```

The recommended way - keeps styles separate from markup.

2. Style element

```
<html>
  <head>
    <style>
      .cool-navbar {
        display: flex;
        color: #343434;
      }
    </style>
  </head>
  <body>
    <nav class="cool-navbar">...</nav>
  </body>
</html>
```

Useful for quick prototypes or critical CSS.

3. Inline styles

```
<nav style="display: flex; color: #343434">...</nav>
```

- Avoid this in production code - no pseudo-classes, no media queries, no reusability.
- In Tailwind, we'll use something that *looks* similar to inline styles but works completely differently.

How CSS works

1. Browser loads HTML and parses it into a DOM tree
2. Fetches linked CSS files
3. Parses CSS, matches selectors to DOM nodes
4. Combines styles (the "cascade")
5. Computes final values for each element
6. Paints pixels on screen

The Cascade

Styles **cascade** - they flow down and combine.

When multiple rules target the same element, CSS decides which wins:

1. **Origin** - browser defaults < your styles < `!important`
2. **Specificity** - more specific selectors win
3. **Source order** - later rules win over earlier ones

```
p {  
  color: red;  
} /* from stylesheet 1 */  
p {  
  color: blue;  
} /* from stylesheet 2 - wins (later) */
```

CSS Anatomy

```
selector {  
  property: value;  
}
```

```
.btn {  
  font-size: 1.15rem;  
  color: white;  
  padding: 1rem 1.5rem;  
  background-color: #36393f;  
}
```

Selector picks which elements to style. **Properties** define how they look.

CSS Selectors

```
/* By element type */
div {
  color: red;
}

/* By class name (most common) */
.card {
  padding: 1rem;
}

/* By ID (use sparingly) */
#main-header {
  font-size: 2rem;
}

/* Combined – div that has class "card" */
div.card {
  border: 1px solid gray;
}
```

Selector combinators

```
/* Descendant – any .title inside .card (any depth) */  
.card .title {  
  font-weight: bold;  
}  
  
/* Direct child – only immediate children */  
.card > .title {  
  font-weight: bold;  
}  
  
/* Multiple selectors – same styles for both */  
.card,  
.badge {  
  border-radius: 8px;  
}
```

Pseudo-classes

```
/* Hover state */
.btn:hover {
  background-color: blue;
}

/* Focus state – when element is focused (keyboard, click) */
input:focus {
  border-color: green;
}

/* Structural pseudo-classes */
li:first-child {
  font-weight: bold;
}
li:last-child {
  border-bottom: none;
}
tr:nth-child(even) {
  background: #f0f0f0;
}
```

Specificity

Who wins when multiple rules match the same element?

element (0,0,1) < **class** (0,1,0) < **id** (1,0,0) < **inline style**

```
div {  
  color: red;  
} /* 0,0,1 */  
.card {  
  color: blue;  
} /* 0,1,0 - wins over element */  
#main {  
  color: green;  
} /* 1,0,0 - wins over class */  
div.card {  
  color: orange;  
} /* 0,1,1 - wins over .card alone */
```

If you need `!important` to fix your styling, something went wrong earlier.

CSS Inheritance

Some CSS properties are **inherited** by children automatically:

```
<div style="color: blue; font-family: Arial;">  
  <p>This text is blue and Arial – inherited!</p>  
  <span>This too!</span>  
</div>
```

This is why setting `font-family` on `body` works for the whole page.

Inherited properties:

`color`, `font-family`, `font-size`, `font-weight`, `line-height`, `text-align`, `visibility` ...

NOT inherited:

`padding`, `margin`, `border`, `background`, `width`, `height`, `display` ...

CSS in React

In React, we use `className` instead of `class` (because `class` is a reserved word in JS):

```
import './Button.css';

function Button() {
  return <button className="btn btn-primary">Click me</button>;
}
```

```
/* Button.css */
.btn {
  padding: 0.5rem 1rem;
  border-radius: 4px;
}
.btn-primary {
  background-color: blue;
  color: white;
}
```

The problem with plain CSS in React

```
/* Button.css */
.btn {
  padding: 1rem;
}

/* Card.css - loaded later */
.btn {
  padding: 2rem;
} /* Overwrites Button's styles! */
```

- CSS has **global scope** - every class name can conflict with every other.
- CSS was designed for documents, not component-based apps.

Checkpoint: CSS Basics

- Three ways to include styles (external, style element, inline)
- Selectors: element, class, id, combinators, pseudo-classes
- Specificity determines which rule wins
- Some properties are inherited by children (`color`, `font-family` ...)
- In React: `className` instead of `class`
- Global CSS scope causes naming conflicts in component apps

Questions?

Styling vs. Layouting

Styling vs. Layouting

Styling = How things *look*

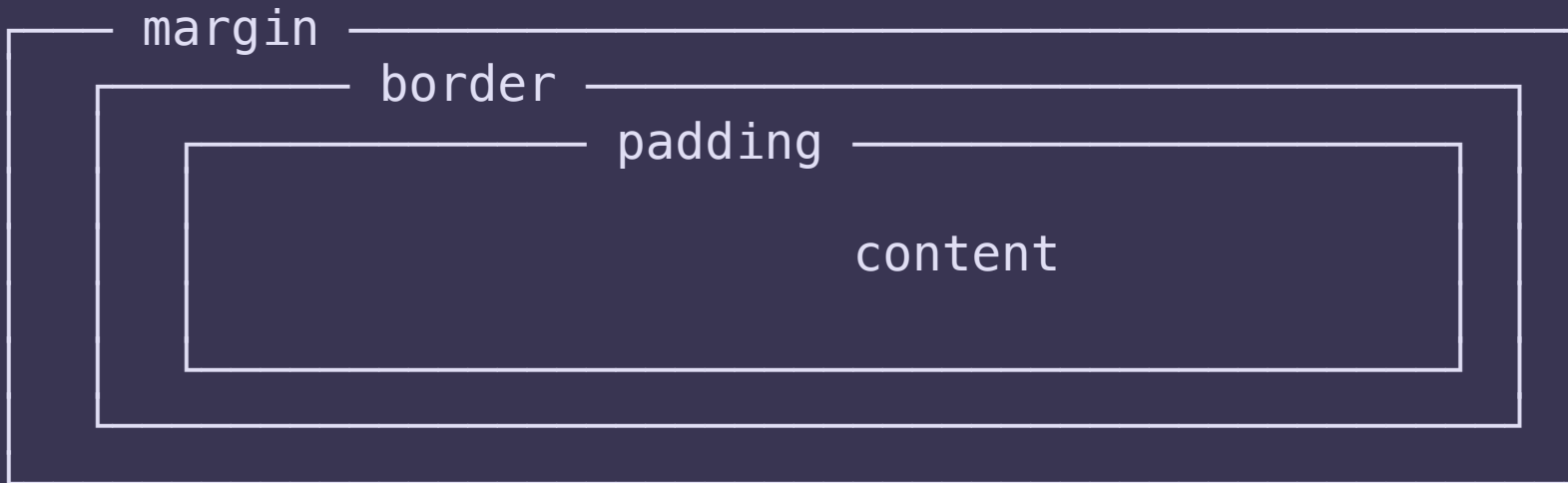
- Colors, fonts, borders, shadows, rounded corners
- Hover effects, transitions
- *"Make the button blue with rounded corners"*

Layouting = Where things *are*

- Positioning elements on the page
- Flexbox, Grid, spacing
- Responsive behavior
- *"Put these 3 cards in a row, stack them on mobile"*

The Box Model

Every HTML element is a rectangular box:



- **Content** - the actual text/image
- **Padding** - space between content and border
- **Border** - the edge of the element
- **Margin** - space between this element and others

CSS Units

```
.element {  
  /* Absolute */  
  width: 200px; /* pixels – avoid for font sizes */  
  
  /* Relative to font */  
  font-size: 2rem; /* 2x root font size (recommended) */  
  font-size: 1.5em; /* 1.5x parent's font size */  
  
  /* Relative to viewport */  
  height: 100vh; /* 100% of viewport height */  
  width: 50vw; /* 50% of viewport width */  
  
  /* Percentage – relative to parent */  
  width: 50%;  
}
```

Prefer `rem` for sizes – it respects user's browser font settings.

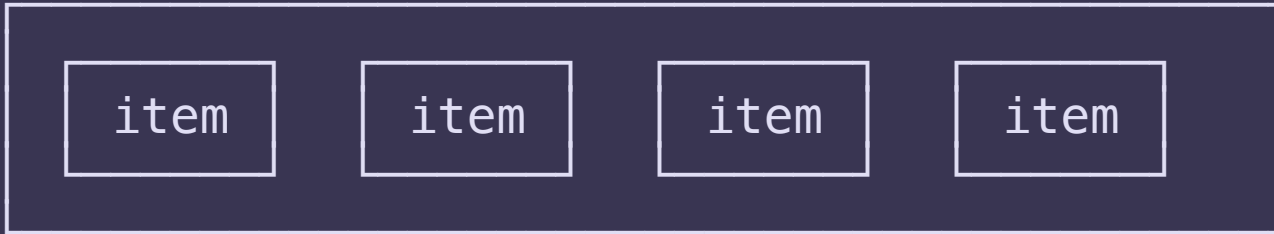
Flexbox

One-dimensional layout - arrange items in a **row** or **column**

Flexbox - the concept

Flexbox creates a **flex container** that controls how its children are arranged:

main axis →



- **Main axis** = the direction items flow (row or column)
- **Cross axis** = perpendicular to main axis
- `justify-content` controls alignment on the **main** axis
- `align-items` controls alignment on the **cross** axis

Flexbox - container properties

```
.container {  
  display: flex; /* enable flexbox */  
  flex-direction: row; /* row (default) | column */  
  flex-wrap: wrap; /* nowrap (default) | wrap */  
  
  /* Main axis alignment */  
  justify-content: flex-start; /* start | center | end */  
  justify-content: space-between; /* equal space between items */  
  justify-content: space-around; /* equal space around items */  
  
  /* Cross axis alignment */  
  align-items: stretch; /* stretch (default) | center */  
  align-items: flex-start; /* start | end | baseline */  
  
  gap: 1rem; /* space between items */  
}
```

Flexbox - item properties

```
.item {  
  flex-grow: 1; /* how much this item grows relative to siblings */  
  flex-shrink: 0; /* how much this item shrinks (0 = don't shrink) */  
  flex-basis: 200px; /* initial size before grow/shrink */  
  
  /* Shorthand */  
  flex: 1; /* grow: 1, shrink: 1, basis: 0% → fill space */  
  flex: 0 0 200px; /* fixed 200px, no grow, no shrink */  
  
  align-self: center; /* override container's align-items for this item */  
}
```

Flexbox - common patterns

```
/* Navbar: logo left, links right */  
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

```
/* Center something perfectly */  
.centered {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```

```
/* Sidebar + content */  
.layout {  
  display: flex;  
}  
.sidebar {  
  flex: 0 0 250px;  
} /* fixed 250px */  
.content {  
  flex: 1;  
} /* fill remaining space */
```

CSS Grid

Two-dimensional layout - **rows AND columns** simultaneously

Grid - the concept

Grid divides a container into rows and columns, creating a grid of cells:

	col 1	col 2	col 3
r1	item 1	item 2	item 3
r2	item 4	item 5	item 6
r3	item 7	item 8	item 9

Items can **span** multiple rows/columns. Great for page layouts and card grids.

Grid - container properties

```
.container {  
  display: grid;  
  
  /* Define columns */  
  grid-template-columns: 200px 1fr 1fr; /* fixed + flexible */  
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); /* responsive! */  
  
  /* Define rows */  
  grid-template-rows: 60px 1fr 40px; /* header, content, footer */  
  
  gap: 1rem; /* space between cells */  
}
```

1fr = one **fraction** of the available space. Think of it as "fill proportionally".

Grid - item properties

```
/* Item spanning multiple columns */
.wide-item {
  grid-column: 1 / 3; /* from column line 1 to 3 (spans 2 cols) */
  grid-column: span 2; /* shorthand: span 2 columns */
}

/* Item spanning multiple rows */
.tall-item {
  grid-row: 1 / 3; /* from row line 1 to 3 */
}

/* Place item at specific position */
.placed-item {
  grid-column: 2;
  grid-row: 1;
}
```

Grid - page layout example

```
.page {  
  display: grid;  
  grid-template-columns: 250px 1fr;  
  grid-template-rows: 60px 1fr 40px;  
  min-height: 100vh;  
}  
  
.header {  
  grid-column: 1 / -1;  
} /* span full width */  
.sidebar {  
  grid-row: 2;  
}  
.content {  
  grid-row: 2;  
}  
.footer {  
  grid-column: 1 / -1;  
} /* span full width */
```

Grid - page layout example

header	
sidebar	content
footer	

Flexbox vs Grid - when to use which?

Flexbox	Grid
One dimension (row OR column)	Two dimensions (rows AND columns)
Content-driven sizing	Layout-driven sizing
Navbar items, button groups	Page layouts, card grids
When items should grow/shrink	When you need precise placement

In practice, you'll use both - often nested. A grid layout might have flex containers inside each cell.

Responsive design

Mobile-first: write base styles for mobile, then add styles for larger screens.

```
/* Base: mobile (single column) */
.grid {
  display: grid;
  grid-template-columns: 1fr;
}

/* Tablet: 2 columns */
@media (min-width: 768px) {
  .grid {
    grid-template-columns: 1fr 1fr;
  }
}
```

Checkpoint: Styling & Layouting

- **Box model**: content → padding → border → margin
- **Flexbox**: one-dimensional layout (row or column), justify-content + align-items
- **Grid**: two-dimensional layout (rows and columns), grid-template-columns/rows
- **Flexbox** for component-level alignment, **Grid** for page-level layouts
- **Responsive**: mobile-first approach with media queries

Questions?

Tailwind CSS

What is Tailwind CSS?

- A **utility-first** CSS framework
- Instead of writing CSS rules, you apply small utility classes directly
- Each class does exactly one thing:
 - `p-4` = padding 1rem,
 - `text-red-500` = red text
- No more switching between HTML and CSS files
- No more inventing class names like `.card-wrapper-inner-container-v2`

Plain CSS vs Tailwind

```
/* Plain CSS */  
.card {  
  padding: 1.5rem;  
  background-color: white;  
  border-radius: 0.5rem;  
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);  
}
```

```
<!-- Tailwind -->  
<div class="p-6 bg-white rounded-lg shadow-sm">...</div>
```

Same result. Zero context switching. No naming headaches.

How does Tailwind work?

1. You write utility classes in your HTML/JSX
2. Tailwind **scans** all your source files for class names
3. It generates **only** the CSS for classes you actually use
4. Dead code elimination built-in - tiny bundle sizes

```
Your code:   className="p-4 text-blue-500 rounded-lg"
              ↓
Tailwind:    Scans files, finds used classes, generates CSS
              ↓
Output CSS:  .p-4 { padding: 1rem; }
             .text-blue-500 { color: #3b82f6; }
             .rounded-lg { border-radius: 0.5rem; }
```

Tailwind in React

```
function Card({ title, description }: { title: string; description: string }) {  
  return (  
    <div className="p-6 bg-white rounded-lg shadow-md">  
      <h2 className="text-xl font-bold text-gray-900">{title}</h2>  
      <p className="mt-2 text-gray-600">{description}</p>  
    </div>  
  );  
}
```

No CSS files needed. No class name collisions. Styles live with the component.

Layouting with Tailwind - Flexbox

Every CSS flex/grid property maps to a Tailwind class:

```
<!-- Horizontal layout with centered items -->
<div class="flex items-center justify-between gap-4">
  <span>Logo</span>
  <nav class="flex gap-2">
    <a href="#">Home</a>
    <a href="#">About</a>
  </nav>
</div>

<!-- Vertical layout -->
<div class="flex flex-col gap-4">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
</div>
```

Flex classes mapping

CSS	Tailwind
<code>display: flex</code>	<code>flex</code>
<code>flex-direction: column</code>	<code>flex-col</code>
<code>justify-content: center</code>	<code>justify-center</code>
<code>justify-content: space-between</code>	<code>justify-between</code>
<code>align-items: center</code>	<code>items-center</code>
<code>flex-wrap: wrap</code>	<code>flex-wrap</code>
<code>gap: 1rem</code>	<code>gap-4</code>
<code>flex: 1</code>	<code>flex-1</code>

Grid classes mapping

CSS	Tailwind
<code>display: grid</code>	<code>grid</code>
<code>grid-template-columns: repeat(3, 1fr)</code>	<code>grid-cols-3</code>
<code>grid-template-rows: repeat(2, 1fr)</code>	<code>grid-rows-2</code>
<code>grid-column: span 2</code>	<code>col-span-2</code>
<code>grid-row: span 2</code>	<code>row-span-2</code>
<code>gap: 1rem</code>	<code>gap-4</code>

Responsive layout with Tailwind

Mobile-first: base classes apply to all sizes, prefixes add rules for larger screens.

```
<div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">  
  <div>Card 1</div>  
  <div>Card 2</div>  
</div>
```

Prefix	Min width	Target
(none)	0px	Mobile (base)
sm:	640px	Large phone
md:	768px	Tablet
lg:	1024px	Desktop
xl:	1280px	Large desktop

Responsive example - Navbar

```
<nav class="flex flex-col md:flex-row items-center justify-between p-4 shadow">
  <h1 class="text-xl font-bold">MyApp</h1>

  <!-- Stack on mobile, row on desktop -->
  <div class="flex flex-col md:flex-row gap-2 md:gap-4 mt-4 md:mt-0">
    <a class="text-gray-600 hover:text-gray-900">Home</a>
    <a class="text-gray-600 hover:text-gray-900">About</a>
    <a class="text-gray-600 hover:text-gray-900">Contact</a>
  </div>
</nav>
```

No media queries. No separate CSS file. Just prefixes that map to `@media` rules.

Styling with Tailwind - Spacing

```
<!-- Padding -->
<div class="p-4">All sides (1rem)</div>
<div class="px-4 py-2">Horizontal 1rem, vertical 0.5rem</div>
<div class="pt-2 pr-4 pb-2 pl-4">Individual sides</div>

<!-- Margin -->
<div class="m-4">All sides</div>
<div class="mx-auto">Center horizontally</div>
<div class="mt-8">Top margin only</div>

<!-- Gap between children (flex/grid) -->
<div class="flex gap-4">
  <div>Child 1</div>
  <div>Child 2</div>
</div>
```

Spacing scale: each unit = **0.25rem** (4px). So `p-4` = 1rem = 16px.

Styling with Tailwind - Colors

```
<!-- Text color -->
<p class="text-gray-900">Dark text</p>
<p class="text-blue-500">Blue text</p>
<p class="text-red-600">Error text</p>

<!-- Background color -->
<div class="bg-white">White bg</div>
<div class="bg-gray-100">Light gray bg</div>
<div class="bg-blue-500">Blue bg</div>

<!-- Border color -->
<div class="border border-gray-300">Gray border</div>
```

Color scale: 50, 100, 200, ..., 800, 900, 950

Lower = lighter, higher = darker.

Styling with Tailwind - Typography & more

```
<!-- Font size & weight -->
<h1 class="text-3xl font-bold">Big bold title</h1>
<p class="text-sm text-gray-500">Small muted text</p>

<!-- Rounded corners -->
<div class="rounded">Slightly rounded (4px)</div>
<div class="rounded-lg">More rounded (8px)</div>
<div class="rounded-full">Pill / circle</div>

<!-- Shadows -->
<div class="shadow-sm">Subtle shadow</div>
<div class="shadow-lg">Dramatic shadow</div>

<!-- Interactive states -->
<button class="bg-blue-500 hover:bg-blue-600 focus:ring-2">Hover me</button>
```

Full styled component example

```
type ProfileCardProps = {
  name: string;
  role: string;
  avatarUrl: string;
};

function ProfileCard({ name, role, avatarUrl }: ProfileCardProps) {
  return (
    <div className="flex items-center gap-4 p-4 bg-white rounded-xl shadow-md">
      <img src={avatarUrl} className="size-12 rounded-full object-cover" />
      <div>
        <h3 className="font-semibold text-gray-900">{name}</h3>
        <p className="text-sm text-gray-500">{role}</p>
      </div>
    </div>
  );
}
```

Tailwind Configuration

Tailwind is customizable via `tailwind.config.js` (v3) or CSS config (v4):

```
export default {
  theme: {
    extend: {
      colors: {
        brand: {
          50: "#eff6ff",
          500: "#3b82f6",
        },
      },
      borderRadius: { "4xl": "2rem" },
    },
  },
};
```

Now you can use `bg-brand-500`, `rounded-4xl` in your markup.

Dark mode & CSS Variables

Modern Tailwind uses CSS variables under the hood:

```
/* Tailwind generates these */
:root {
  --color-blue-500: #3b82f6;
  --radius-lg: 0.5rem;
}
```

Dark mode is built-in:

```
<div class="bg-white dark:bg-gray-900 text-gray-900 dark:text-white">
  Adapts automatically to user's system preference!
</div>
```

VS Code tip

Install the **Tailwind CSS IntelliSense** extension.

- Autocomplete for all Tailwind classes
- Hover preview showing the actual CSS a class generates
- Color swatches displayed inline

You don't memorize classes.

Type what you want (`bg-`, `text-`, `flex-`, `p-`) and let IntelliSense help you.

Checkpoint: Tailwind CSS

- Utility-first: one class = one CSS property
- Tailwind scans your files and generates only CSS you use
- Flex/Grid layout classes map directly to CSS properties
- Responsive: mobile-first with `sm:`, `md:`, `lg:` prefixes
- Styling: spacing (`p-`, `m-`), colors (`text-`, `bg-`), typography, shadows, borders
- Configuration: custom colors, fonts, spacing via config file

Questions?

Tailwind Utilities

Why `cn()`, `clsx`, and `twMerge` exist

The className problem

Conditional classes get messy with string concatenation:

```
const Button = () => {  
  return (  
    <button  
      className={  
        "px-4 py-2 rounded-lg font-medium " +  
        (variant === "primary" ? "bg-blue-500 text-white " : "") +  
        (variant === "secondary" ? "bg-gray-200 text-gray-800 " : "") +  
        (disabled ? "opacity-50 cursor-not-allowed " : "") +  
        (size === "lg" ? "text-lg px-6 py-3 " : "")  
      }  
    ></button>  
  );  
};
```

Fragile, produces trailing spaces, hard to read.

clsx - conditional class names

```
import { clsx } from "clsx";

<button className={clsx(
  "px-4 py-2 rounded-lg font-medium",
  variant === "primary" && "bg-blue-500 text-white",
  variant === "secondary" && "bg-gray-200 text-gray-800",
  disabled && "opacity-50 cursor-not-allowed",
  size === "lg" && "text-lg px-6 py-3",
)}>
```

clsx ignores undefined, false, null - clean conditional classes.

tailwind-merge - smart class deduplication

Problem: what happens with conflicting Tailwind classes?

```
clsx("p-4 text-red-500", "px-6 text-blue-500");  
// → "p-4 text-red-500 px-6 text-blue-500"  
// Both text colors are in the string! Browser uses last one... maybe.
```

tailwind-merge understands Tailwind's class hierarchy:

```
import { twMerge } from "tailwind-merge";  
  
twMerge("p-4 text-red-500", "px-6 text-blue-500");  
// → "p-4 px-6 text-blue-500"  
// Correctly removes conflicting classes, last wins
```

The `cn()` utility

Every serious Tailwind project has this little helper:

```
// lib/utils.ts
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

Usage - conditionals + deduplication in one call

```
cn("px-4 py-2 rounded-lg", variant === "primary" && "bg-blue-500", className);
```

This is the standard. Shadcn includes it by default.

CSS Approaches Compared

Comparison

Approach	Pros	Cons
Raw CSS	Full control, no deps	Global scope, naming issues, file switching
CSS Modules	Scoped styles, no collisions	Still writing CSS files, context switching
CSS-in-JS	JS variables, scoped	Runtime overhead, bundle size
Tailwind	No CSS files, tiny bundle, fast	Long classNames, initial learning curve

CSS-in-JS example

```
// styled-components / Emotion
const Button = styled.button`
  padding: 8px 16px;
  background-color: ${props => (props.$primary ? "blue" : "gray")};
  border-radius: 4px;
  &:hover {
    opacity: 0.8;
  }
`;

<Button $primary>Click me</Button>;
```

Why it's falling out of favor:

- **Runtime cost** - styles computed in JS at render time
- **Bundle size** - ships the entire styling library to the client
- Major libraries (MUI, Chakra) are moving away from it

Our recommendation

Use **Tailwind CSS** for this course and your projects.

- Industry standard in 2025/2026
- Zero runtime cost
- Massive ecosystem and community
- Works perfectly with React and component libraries
- You'll use it with Shadcn in your project

Checkpoint: Utilities & CSS Approaches

- `clsx` - conditional class name composition
- `tailwind-merge` - resolves conflicting Tailwind classes
- `cn()` = `twMerge(clsx(...))` - the standard combo
- CSS-in-JS has runtime overhead and is losing popularity
- Raw CSS has global scope problems in component apps
- Tailwind is the modern standard - zero runtime, utility-first

Questions?

Component Libraries

Why component libraries?

Building a good `<Dialog>` from scratch requires:

- Focus trapping (Tab key stays inside the dialog)
- Escape key to close
- Screen reader announcements (ARIA attributes)
- Scroll locking on the body behind the dialog
- Animation on open/close
- Click-outside to close

That's ~600 lines for ONE component. And you'll still have accessibility bugs.

Headless UI libraries

Headless = all the behavior and accessibility, zero styling.

- **Radix UI** - accessible primitives (Dialog, Dropdown, Tabs, Tooltip...)
- They handle keyboard navigation, ARIA, focus management
- You bring your own styles (e.g., Tailwind)

```
<!-- Radix - you get behavior, you style it yourself -->
<Dialog.Root>
  <Dialog.Trigger class="...">Open</Dialog.Trigger>
  <Dialog.Content class="...">
    <Dialog.Title>Are you sure?</Dialog.Title>
    <Dialog.Close class="...">Cancel</Dialog.Close>
  </Dialog.Content>
</Dialog.Root>
```

From headless to styled: Shadcn

Shadcn is NOT a npm dependency. It's a collection of **copy-paste components** built on:

- **Radix UI** or **Base UI** for accessibility and logic
- **Tailwind CSS** for styling
- **cva** (class-variance-authority) for variant management

The code lives **in your project**. You own it. You can customize everything.

```
pnpm dlx shadcn@latest init
pnpm dlx shadcn@latest add button card input dialog
```

Components appear in `src/components/ui/` - they're your files now.

Shadcn Button - how it actually works

Shadcn uses `cva` (class-variance-authority) to define variants:

```
const buttonVariants = cva(
  "inline-flex items-center justify-center rounded-md text-sm font-medium transition-colors",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground hover:bg-primary/90",
        outline: "border border-input bg-background hover:bg-accent",
        ghost: "hover:bg-accent hover:text-accent-foreground",
      },
      size: {
        default: "h-10 px-4 py-2",
        sm: "h-9 rounded-md px-3",
        lg: "h-11 rounded-md px-8",
      },
    },
    defaultVariants: { variant: "default", size: "default" },
  },
);
```

Shadcn Button - the component

```
import { type ButtonHTMLAttributes, forwardRef } from "react";

const Button = forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, ...props }, ref) => (
    <button
      className={cn(buttonVariants({ variant, size, className }))}
      ref={ref}
      {...props}
    />
  ),
);
```

`cva` generates the right classes based on variant + size props. `cn` merges them with any `className` override from the parent.

Using Shadcn components

```
import { Button } from "@components/ui/button";
import { Card, CardHeader, CardTitle, CardContent } from "@components/ui/card";
import { Input } from "@components/ui/input";

function LoginForm() {
  return (
    <Card className="w-96 mx-auto mt-10">
      <CardHeader>
        <CardTitle>Sign In</CardTitle>
      </CardHeader>
      <CardContent className="flex flex-col gap-4">
        <Input placeholder="Email" type="email" />
        <Input placeholder="Password" type="password" />
        <Button>Log In</Button>
        <Button variant="outline">Sign Up Instead</Button>
      </CardContent>
    </Card>
  );
}
```

Why Shadcn?

- **Accessible by default** - built on Radix, keyboard-navigable, screen reader friendly
- **Customizable** - it's your code, change anything
- **Tailwind native** - uses `cn()`, `cva`, and Tailwind classes
- **No dependency lock-in** - doesn't update under you unexpectedly
- **Beautiful defaults** - looks professional out of the box
- Used by Vercel, Cal.com, and a large part of the React ecosystem

Checkpoint: Component Libraries & Shadcn

- Don't build complex accessible components from scratch
- **Headless libraries** (Radix) = behavior + accessibility, no styling
- **Shadcn** = Radix + Tailwind + cva, copy-pasted into your project
- `cva` manages variant styles cleanly (variant, size, etc.)
- Components live in YOUR codebase - full control

Questions?

React Component Patterns

Component anatomy

A React component is a function that takes props and returns JSX:

```
type GreetingProps = {  
  name: string;  
};  
  
function Greeting({ name }: GreetingProps) {  
  return <h1>Hello, {name}!</h1>;  
}  
  
// Usage  
<Greeting name="PB138" />;
```

Props go in, JSX comes out.

The `children` prop

`children` is a special prop - it's whatever you put **between** the tags:

```
type CardProps = {
  children: React.ReactNode;
};

function Card({ children }: CardProps) {
  return <div className="p-4 bg-white rounded-lg shadow">{children}</div>;
}

// Usage - anything can go inside
<Card>
  <h2>Title</h2>
  <p>Some content here</p>
  <Button>Action</Button>
</Card>;
```

This is **composition** - the Card doesn't know or care what's inside it.

Slots pattern

When you need more control than just `children`, use multiple props as "slots":

```
type PageLayoutProps = {
  header: React.ReactNode;
  sidebar: React.ReactNode;
  children: React.ReactNode;
};

const PageLayout = ({ header, sidebar, children }: PageLayoutProps) => (
  <div className="min-h-screen grid grid-cols-[250px_1fr] grid-rows-[60px_1fr]">
    <header className="col-span-2 bg-white shadow">{header}</header>
    <aside className="bg-gray-50 p-4">{sidebar}</aside>
    <main className="p-6">{children}</main>
  </div>
)

<PageLayout header={<Navbar />} sidebar={<SideMenu />}>
  {/* children */}
</PageLayout>;
```

Composition over configuration

The "god component" anti-pattern - too many props:

```
<DataDisplay
  showHeader={true}
  headerTitle="Users"
  showSearch={true}
  showFilters={true}
  filterOptions={['active', 'inactive']}
  columns={[]}
  data={users}
  onClick={handleClick}
  showPagination={true}
  pageSize={10}
  // ... 20 more props
/>
```

This gets impossible to maintain and test.

Composition over configuration

Composed from smaller, focused pieces:

```
<Card>
  <CardHeader>
    <CardTitle>Users</CardTitle>
    <SearchInput value={search} onChange={setSearch} />
  </CardHeader>
  <CardContent>
    <FilterBar options={["active", "inactive"]} />
    <DataTable columns={columns} data={users} />
  </CardContent>
  <CardFooter>
    <Pagination pageSize={10} total={users.length} />
  </CardFooter>
</Card>
```

Each piece is small, testable, reusable, and has a clear responsibility.

Refactoring example - before

A 70-line "god component" with everything inlined:

```
const Dashboard = () => (
  <div className="p-6">
    <div className="flex justify-between mb-6">
      <h1 className="text-2xl font-bold">Dashboard</h1>
      <button className="px-4 py-2 bg-blue-500 text-white rounded">
        Add New
      </button>
    </div>
    <div className="grid grid-cols-3 gap-4 mb-6">
      <div className="p-4 bg-white rounded-lg shadow">
        <p className="text-sm text-gray-500">Total Users</p>
        <p className="text-2xl font-bold">1,234</p>
      </div>
      {/* ... 50 more lines of the same ... */}
    </div>
  </div>
);
```

Refactoring example - after

Extract repeated patterns into components:

```
const StatCard = ({ label, value }: { label: string; value: string }) => (  
  <div className="p-4 bg-white rounded-lg shadow">  
    <p className="text-sm text-gray-500">{label}</p>  
    <p className="text-2xl font-bold">{value}</p>  
  </div>  
);  
  
const Dashboard = () => (  
  <div className="p-6">  
    <DashboardHeader onAdd={handleAdd} />  
    <div className="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">  
      <StatCard label="Total Users" value="1,234" />  
      <StatCard label="Revenue" value="$12,345" />  
    </div>  
    <DataTable data={recentActivity} />  
  </div>  
);
```

Checkpoint: React Component Patterns

- Components = functions that take props and return JSX
- `children` prop for flexible composition
- Slots pattern for multiple content areas
- **Composition over configuration** - small focused components > one god component
- Extract when you see repeated patterns or a component gets too long

Questions?

State & Effects

Why do we need `useState`?

What happens if you use a regular variable?

```
function Counter() {  
  let count = 0;  
  
  const handleClick = () => (count += 1);  
  
  return (  
    <div>  
      <span>{count}</span>  
      <button onClick={handleClick}>Add</button>  
    </div>  
  );  
}
```

Clicking the button updates `count` in memory, but **React doesn't know about it**. The component never re-renders. The UI stays at 0 forever.

useState - triggering re-renders

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div className="flex items-center gap-4">
      <span className="text-xl font-bold">{count}</span>
      <Button onClick={() => setCount(count + 1)}>+</Button>
    </div>
  );
}
```

1. A **value** that persists across renders
2. A **setter function** that tells React to re-render with the new value

Regular variables get reset every render. State survives.

useState - form example

```
function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [isLoading, setIsLoading] = useState(false);

  const handleSubmit = () => {
    setIsLoading(true);
    // ... submit logic
  };

  return (
    <form onSubmit={handleSubmit} className="flex flex-col gap-4">
      <Input
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <Input
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        type="password"
      />
      <Button disabled={isLoading}>
        {isLoading ? "Signing in..." : "Sign In"}
      </Button>
    </form>
  );
}
```

useState - gotchas

```
// State updates are asynchronous and batched
setCount(count + 1);
setCount(count + 1); // Still uses the OLD count value!

// Use the function form when updating based on previous state
setCount((prev) => prev + 1);
setCount((prev) => prev + 1); // Correctly increments twice

// Don't mutate objects directly - React won't detect the change
const [user, setUser] = useState({ name: "Alice", age: 25 });
user.name = "Bob"; // React doesn't notice this!

// Create a new object instead
setUser({ ...user, name: "Bob" }); // New object → React re-renders
```

useEffect - side effects

useEffect runs code **after** the component renders.

Use it for: subscriptions, timers, DOM manipulation, event listeners.

```
import { useState, useEffect } from "react";

function WindowSize() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener("resize", handleResize);

    // Cleanup: runs when component unmounts or before re-running effect
    return () => window.removeEventListener("resize", handleResize);
  }, []); // Empty array = run once on mount

  return <p>Window width: {width}px</p>;
}
```

useEffect - dependency array

```
// Runs on EVERY render (usually not what you want)
useEffect(() => {
  console.log("I run all the time");
}, []);
```

```
// Runs ONCE after first render (mount)
useEffect(() => {
  console.log("I run once on mount");
}, []);
```

```
// Runs when `count` changes
useEffect(() => {
  document.title = `Count: ${count}`;
}, [count]);
```

```
// Cleanup runs before next effect or on unmount
useEffect(() => {
  const id = setInterval(() => tick(), 1000);
  return () => clearInterval(id);
}, []);
```

useEffect - NOT for data fetching

Using `useEffect` for API calls is a common **anti-pattern**:

```
// Don't do this in production
useEffect(() => {
  fetch("/api/users")
    .then((res) => res.json())
    .then(setUsers);
}, []);
```

Problems: no loading states, no error handling, no caching, no deduplication, race conditions.

Use a **data fetching library** instead:

- **TanStack Query** (React Query) - the standard
- **SWR** - lighter alternative

We'll cover data fetching properly in a later week.

useEffect - good use cases

```
// Sync document title with state
useEffect(() => {
  document.title = `${unreadCount} Messages`;
}, [unreadCount]);

// Add/remove event listeners
useEffect(() => {
  const handler = (e: KeyboardEvent) => {
    if (e.key === "Escape") onClose();
  };
  document.addEventListener("keydown", handler);
  return () => document.removeEventListener("keydown", handler);
}, [onClose]);

// Sync with external system (localStorage, WebSocket, etc.)
useEffect(() => {
  localStorage.setItem("theme", theme);
}, [theme]);
```

Checkpoint: State & Effects

- Regular variables don't trigger re-renders - use `useState`
- `useState` returns `[value, setter]` - setter triggers re-render
- Use function form `setState(prev => ...)` when updating based on previous state
- Don't mutate objects - create new ones with spread `{ ...obj, key: value }`
- `useEffect` = run code after render (subscriptions, DOM, timers)
- Dependency array controls when the effect re-runs
- **Don't use `useEffect` for data fetching** - use TanStack Query or SWR

Questions?

CSS, Tailwind & React Components

Petr Wehrenberg